

Exercise 5

Part 1 – Setting up and Refactoring:

Why was it a good idea to parameterize the Triangle tests: Parameterizing the Triangle tests was a good idea because all tests verify the same functionality with different inputs. Parameterized tests reduce code duplication, improve readability, simplify maintenance, and make it easy to add new test cases while clearly expressing the intended behavior of the method.

Part 2 – Implement the New Class:

Task 1 – Test Level Definition:

Look up the definition for the following terms in the ISTQB-Glossary:

Component test: Testing of individual components (units) in isolation from the rest of the system to verify that each behaves as specified.

Integration test: Testing in which software components are combined and tested as groups to detect interface defects and interaction problems.

System test: Testing conducted on a complete, integrated system to evaluate whether it meets specified requirements.

Acceptance test: Formal testing conducted to determine whether a system satisfies business or user requirements and whether it is acceptable for delivery.

Part 2 and Part 3 are done when we are finished with the rest

Task 2 – Test Level Comparison:

Criterion	Component Test	Integration Test	System Test	Acceptance Test
Test objectives	Verify individual components behave according to specification	Verify interactions between components; detect interface and integration defects	Verify that the complete system meets specified requirements	Verify the system meets user needs and business objectives; determine acceptability
Test basis	Component specifications, detailed design	Interface specifications, architecture, component design	System requirements specification	User requirements, business requirements, acceptance criteria
Typical test objects	Single classes, functions, modules	Groups of components, subsystems	Entire system	Complete system from a user/business perspective
Test tools	Unit testing frameworks (e.g., JUnit), code analyzers, stubs/drivers	Integration frameworks, test harnesses, simulators	Test management tools, automation frameworks	End-user testing tools, acceptance testing frameworks
Test environment	Controlled, often simulated (may use stubs/mocks)	Partially integrated environment; may simulate missing components	Production-like environment, fully integrated system	Realistic operational environment; may be production-like or beta environment
Typical failures	Logic errors, calculation errors, boundary errors in components	Incorrect interactions, data mismatches, interface errors	Missing/incorrect functionality, performance, reliability, security issues	Does not meet business/user requirements, usability problems, workflow gaps
Notes	Fast, low cost, early detection of defects	Detects integration issues missed at unit level	Validates the system as a whole; often done before release	Often formal; may involve customer sign-off; ensures business value